

Stage 2 — Musterhaushalte

`gettsim-personas`

Where You Are



Yesterday you changed the law and (hopefully) numbers moved.

Today you vary a household along one dimension and watch the whole budget constraint move.

The Problem Personas Solve

GETTSIM needs **close to a hundred input columns** to compute everything it covers.

If you only care about the income tax, you should not have to invent an employment biography to get a number out.

A persona gives you:

- a **minimal set of inputs** for one question,
- with the nodes you do not care about **overridden** — e.g. pension benefits set to zero when you are looking at the income tax of a young adult,
- plus sensible **tax-transfer targets**.

What a Persona Is

Four things, ready to hand to `main` :

```
from gettsim_personas import grundsicherung_für_erwerbsfähige

persona = grundsicherung_für_erwerbsfähige.SingleAdult(policy_date_str="2023-07-01")

persona.description      # what it is for, and what was zeroed out
persona.input_data_tree  # the example input data
persona.tt_targets_tree  # the TT targets
persona.policy_date
```

Read `persona.description` before you use one. It tells you what its purpose is.

What Exists

module	personas
<code>einkommensteuer_sozialabgaben</code>	<code>Couple1Child</code>
<code>gesetzliche_altersrente</code>	<code>SingleWithFixedPublicPension</code> , <code>CoupleWithFixedPublicPension</code>
<code>grundsicherung_für_erwerbsfähige</code>	<code>SingleAdult</code> , <code>Single1Child</code> , <code>CoupleNoChildren</code> , <code>Couple1Child</code> , <code>Couple2Children</code> , <code>Couple1ChildInKarenzzeit</code>
<code>grundsicherung_im_alter</code>	<code>SingleNoChild</code> , <code>Single1Child</code> , <code>CoupleNoChild</code> , <code>Couple1Child</code>

If none fits your topic, either

- write a new persona (and create a PR such that others can use it – don't worry, it's easy), or
- start from an existing persona and manipulate its input data tree.

Varying a Household

Varying earnings is built in:

```
Single = grundsicherung_für_erwerbsfähige.SingleAdult

swept = Single(
    policy_date_str=POLICY_DATE,
    bruttolohn_m_linspace_grid=Single.LinspaceGrid(
        p0=Single.LinspaceRange(bottom=0, top=2_000),
        n_points=201,
    ),
)
```

Anything else — rent, age, pension income — goes through `upsert_input_data`:

```
persona.upsert_input_data(
    input_data_to_upsert={"wohnen": {"bruttokaltmiete_m_hh": YOUR_INPUT_DATA_HERE}}
)
```

`gettsim-personas` scales all other inputs to match your upserted input data.

Task

09:15–10:20

1. Open [notebooks/02_personas.ipynb](#) .
2. Paste your reform into the marked cell near the top.
3. Pick your persona. If none fits, write one.
4. Choose the dimension your reform actually varies along — earnings is the default, but rent, age, or pension income may be the right one.
5. Run the grid under both environments. Plot them together.

Before the Break

Start this now, then start with your persona work — it runs in the background:

```
$ pixi run pytask -c soep-preparation/pyproject.toml
```

From the repo root, and with the `-c` flag — without it, nothing gets built.

- It needs the raw SOEP `.dta` files flat in `soep-preparation/data/V41/`.
- It writes to `soep-preparation/bld/`, which Friday's 10:50 session needs.
- It takes a while — that is why it runs while you work and through the break.
- If it errors, approach me or Hans-Martin.